

```

// JARVIS mobile - UI, iPhone brain, optional Mac link.
import { Brain, MODELS, Memory, route, runLocal, systemPrompt, Sentences } from '
import { generate, KINDS, unlockAudio } from './make.js';

const $ = (id) => document.getElementById(id);
const esc = (s) => String(s ?? '').replace(/[\<>"]/g, (c) => ({ '&': '&amp;', '<
const nf = (v, d = 2) => (v == null || !isFinite(v)) ? '-' : Number(v).toLocaleSt
const store = { get(k, d) { try { const v = localStorage.getItem(k); return v ==
const params = new URLSearchParams(location.search);

let token = store.get('jarvisToken', null);
let mode = store.get('jarvisMode', 'auto');
let modelChoice = store.get('jarvisModel', 'smart');
let es = null, macOnline = false, macTurn = false, confirmId = null, level = 0, u
let rec = null, listening = false;
const brain = new Brain();
const history = [];
let abort = null;
let macHost = false;           // true only when this page is served by the Mac ap
let lastMade = null;           // last file generated on the phone
const made = [];               // files generated in this session

// ----- brain
function setBrainChip() {
  const c = $('brainChip');
  if (brain.ready) { c.textContent = `GEHIRN · ${brain.kind === 'webgpu' ? 'GPU'
  else { c.textContent = 'GEHIRN AUS'; c.classList.remove('on'); }
  $('engineInfo').textContent = brain.ready ? `Aktiv: ${brain.model} (${brain.kin
  renderOnboarding();
}
let loading = false;
function showProgress(p, txt) {
  const pct = `${Math.max(2, Math.round(p * 100))}%`;
  $('progBar').style.width = pct; $('progTxt').textContent = txt.slice(0, 120);
  const ob = $('obProg'); if (ob) { ob.style.width = pct; $('obTxt').textContent
}
async function loadBrain() {
  if (brain.ready || loading) return;
  loading = true;
  $('loadBrain').disabled = true;
  const ob = $('obLoad'); if (ob) { ob.disabled = true; ob.textContent = 'Lädt ...
  showProgress(0.01, 'Starte ...');
  try {

```

```

try { await Promise.race([navigator.storage?.persist?.(), new Promise((r) =>
if (params.get('engine') === 'mock') {
  brain.engine = { mock: true }; brain.kind = 'webgpu'; brain.model = 'mock';
  // test brain (no download): echoes chat, returns well-formed content for t
  const MOCK = {
    Präsentation: 'TITEL: Testpräsentation\nUNTERTITEL: Erstellt vom Test-Geh
    Dokument: 'TITEL: Testdokument\n# Einleitung\nDies ist ein Absatz mit Uml
    Tabelle: 'TITEL: Städte\nStadt;Einwohner;Bundesland\nBerlin;3755000;Berli
    Website: 'TITEL: Café Morgenrot\nUNTERTITEL: Der beste Kaffee der Stadt.\n
    TikTok: 'TITEL: Kaffee-Fakten\nHOOK: Das wusstest du nicht über Kaffee\n#
  };
  brain.chat = async (msgs, { onDelta = () => {} } = {}) => {
    const last = msgs[msgs.length - 1].content;
    const key = Object.keys(MOCK).find((k) => last.includes('genau diesem For
    const t = key ? MOCK[key] : `Ich bin JARVIS auf deinem iPhone. Du sagtest
    let s = ''; for (const w of t.split(' ')) { s += w + ' '; onDelta(s); awa
  };
} else {
  await brain.load(modelChoice, showProgress);
}
store.set('jarvisBrainLoaded', '1');
$('progTxt').textContent = 'Bereit.';
say('Gehirn geladen. Ich laufe jetzt direkt auf deinem iPhone.');
```

```

} catch (e) {
  const msg = 'Das Gehirn konnte nicht geladen werden: ' + (e.message || e);
  $('progTxt').textContent = msg;
  $('reply').textContent = msg;
  loading = false; setBrainChip();
  const t = $('obTxt'); if (t) t.textContent = msg + ' - Tipp: Im Zahnrad „Schn
  $('loadBrain').disabled = false;
  return;
}
loading = false; $('loadBrain').disabled = false; setBrainChip();
}
function renderOnboarding() {
  const el = $('onboard');
  if (brain.ready) { el.innerHTML = ''; return; }
  if (loading && $('obLoad')) return;
  const wasLoaded = store.get('jarvisBrainLoaded', '0') === '1';
  el.innerHTML = `<div class="onb"><b>${wasLoaded ? 'Gehirn starten' : 'Gehirn au
    <button class="btn hot" id="obLoad" style="width:100%">${wasLoaded ? 'Starten
  const go = (e) => { if (e) e.preventDefault(); try { unlockSpeech(); } catch (e
  $('obLoad').addEventListener('click', go);
}
// ----- Mac link (opt

```

```

async function api(path, body) {
  const r = await fetch(path, { method: body ? 'POST' : 'GET', headers: { 'Content-Type': 'application/json' } });
  if (r.status === 401) { token = null; store.set('jarvisToken', ''); setMac(false); return r.json(); }
}

function setMac(on) {
  macOnline = on;
  const c = $('#macChip'); c.textContent = on ? 'MAC ✓' : (token ? 'MAC OFFLINE' : 'MAC UNKNOWN');
  const relevant = macHost || !!token;
  c.hidden = !relevant; $('#macSection').hidden = !relevant;
  document.querySelectorAll('#mode button[data-m="mac"]').forEach((b) => { b.hidden = !relevant; });
  $('#macInfo').textContent = on ? 'Mac verbunden - schwere Aufgaben laufen dort.' : 'Mac nicht verbunden';
}

function connectMac() {
  if (!token || !macHost) { setMac(false); return; }
  if (es) es.close();
  try { es = new EventSource('/api/events?t=' + encodeURIComponent(token)); } catch (e) {}
  es.onopen = () => setMac(true);
  es.onerror = () => { setMac(false); };
  es.onmessage = (e) => { try { fromMac(JSON.parse(e.data)); } catch (err) { console.log(err); } }
}

$('#pairBtn').onclick = async () => {
  $('#pairErr').textContent = '';
  try {
    const r = await fetch('/api/pair', { method: 'POST', headers: { 'Content-Type': 'application/json' } });
    const j = await r.json();
    if (!j.ok) { $('#pairErr').textContent = j.error || 'Fehlgeschlagen'; return; }
    token = j.token; store.set('jarvisToken', token); connectMac(); $('#pairErr').textContent = '';
  } catch (e) { $('#pairErr').textContent = 'Mac nicht erreichbar.'; }
};

document.addEventListener('visibilitychange', () => { if (!document.hidden && token) connectMac(); });

let macReply = '';
function fromMac(m) {
  switch (m.type) {
    case 'state': if (macTurn) setState(m.state, m.detail); break;
    case 'reply':
      if (!macTurn) break;
      if (m.reset) macReply = '';
      if (m.delta) macReply += m.delta; else if (m.text != null) { macReply = m.text; }
      $('#reply').textContent = macReply.replace(/\s+/g, ' ').trim();
      if (m.done) { if (macReply.trim()) speak(macReply, true); macTurn = false; break; }
    case 'speak': if (macTurn) speak(m.text, false); break;
    case 'view': if (macTurn) renderView(m.view, m.data || {}); break;
    case 'code': if (macTurn && m.op === 'end' && m.path) renderView('code', { path: m.path }); break;
  }
}

```

```

    case 'telemetry': lastTele = m; if (curView === 'core') renderView('core', {})
    case 'confirm': confirmId = m.id; $('cfK').textContent = ({ destructive: 'ACH
    case 'confirm_close': $('confirm').hidden = true; confirmId = null; break;
  }
}
$('cfYes').onclick = () => { api('/api/confirm', { id: confirmId, accept: true })
$('cfNo').onclick = () => { api('/api/confirm', { id: confirmId, accept: false })
$('macMute').onchange = () => api('/api/mute', { value: $('macMute').checked }).c

// ----- command flow
const STATE = { sleeping: 'Bereit', listening: 'Hört zu', thinking: 'Denkt nach',
function setState(s, detail) { uiState = s; $('state').textContent = detail || ST
function say(text) { $('reply').textContent = text; speak(text, true); }

async function send(text) {
  text = (text || '').trim();
  if (!text) return;
  unlockSpeech();
  $('you').textContent = text; $('text').value = ''; $('reply').textContent = '';
  if (abort) abort.abort();
  stopSpeaking();
  const r = route(text, { macAvailable: macOnline && mode !== 'phone' });
  const toMac = (mode === 'mac' && macOnline) || (r && r.mac);
  if (toMac) {
    macTurn = true; macReply = ''; setState('thinking', 'Mac arbeitet ...');
    try { await api('/api/command', { text }); } catch (e) { macTurn = false; set
    return;
  }
  if (r && r.reply) { setState('speaking'); say(r.reply); remember(text, r.reply)
  if (r && r.gen) { await makeOnPhone(r.gen, r.topic, text); return; }
  if (r && r.local === 'share') {
    if (!lastMade) { say('Ich habe noch nichts erstellt, das ich teilen könnte. S
    renderMade(lastMade, true);
    say(lastMade.kind === 'video' ? 'Tippe auf „Teilen“ und wähle TikTok - die Be
    return;
  }
  if (r && r.local) {
    setState('thinking', 'Lokal ...');
    try {
      const res = await runLocal(r.local, r.args);
      if (res.view) renderView(res.view, res.data || {});
      say(res.say); remember(text, res.say);
    } catch (e) {
      if (macOnline && mode !== 'phone') { macTurn = true; setState('thinking', '
      say('Das hat gerade nicht geklappt: ' + (e.message || e));
    }
  }
}

```

```

    setState('sleeping');
    return;
}
// free conversation → the brain on the phone (or the Mac as fallback)
if (!brain.ready) {
    if (macOnline) { macTurn = true; setState('thinking', 'Mac denkt ...'); api('/a
    say('Lade zuerst das Gehirn - oben auf „Jetzt laden“ tippen.');
```

 return;
 }
 setState('thinking');
 abort = new AbortController();
 const sent = new Sentences();
 const messages = [{ role: 'system', content: systemPrompt(store.get('jarvisTitl
 try {
 const full = await brain.chat(messages, {
 signal: abort.signal,
 onDelta: (t) => { \$('reply').textContent = t; if (uiState !== 'speaking') s
 });
 for (const s of sent.take(full, true)) speak(s, false);
 remember(text, full);
 } catch (e) { say('Fehler im Gehirn: ' + (e.message || e)); }
 setState('sleeping');
}
// ----- files made on
async function makeOnPhone(kind, topic, text) {
 if (!brain.ready) {
 if (macOnline && mode !== 'phone') { macTurn = true; setState('thinking', 'Ma
 say('Dafür brauche ich das Gehirn - tippe oben auf „Jetzt laden“.); return;
 }
 if (brain.busy) { say('Einen Moment, ich bin noch mit der letzten Aufgabe besch
 const label = KINDS[kind].label;
 setState('thinking', `\${label} ...`);
 speak(`Sehr wohl. Ich erstelle \${kind === 'video' ? 'das Video' : kind === 'htm
 curView = 'making';
 \$('view').innerHTML = `

```

    setState('sleeping');
}
function renderMade(m, highlightShare = false) {
    curView = 'made';
    const s = m.spec || {};
    let body = '';
    if (m.kind === 'pptx') body = `<div class="slides"><div><b>${esc(s.title)}</b>`;
    else if (m.kind === 'pdf' || m.kind === 'docx') body = `<div class="list">${(s.
    else if (m.kind === 'xlsx') body = `<div style="overflow-x:auto;margin-top:8px"
    else if (m.kind === 'html') body = `<iframe class="site" sandbox="" title="Vors
    else if (m.kind === 'video') body = `<video src="${m.url}" controls playsinline
    $('view').innerHTML = `<div class="card"><h3>${esc(KINDS[m.kind].label.toUpperC
        <div class="acts2"><button class="btn ${highlightShare || m.kind === 'video'
        ${made.length > 1 ? `<div class="list" style="margin-top:12px">${made.slice(1
    if (m.kind === 'html') $('view').querySelector('iframe').srcdoc = m.html;
    $('view').scrollIntoView({ behavior: 'smooth', block: 'start' });
    $('mkShare').onclick = () => shareFile(m);
    $('mkOpen').onclick = () => window.open(m.url, '_blank');
    const cap = $('mkCap');
    if (cap) cap.onclick = () => navigator.clipboard?.writeText(`${s.caption || ''}
    $('view').querySelectorAll('[data-made]').forEach((a) => (a.onclick = (e) => {
}
async function shareFile(m) {
    const data = { files: [m.file], title: m.file.name };
    try {
        if (navigator.canShare && navigator.canShare(data)) { await navigator.share(d
    } catch (e) { if (e.name === 'AbortError') return; }
    const a = document.createElement('a'); a.href = m.url; a.download = m.file.name
}

function remember(user, assistant) {
    history.push({ role: 'user', content: user }, { role: 'assistant', content: ass
    while (history.length > 16) history.shift();
}

$('text').addEventListener('keydown', (e) => { if (e.key === 'Enter') send($('tex
document.querySelectorAll('.chip').forEach((b) => (b.onclick = () => {
    if (b.dataset.fill) { const t = $('text'); t.value = b.dataset.fill; t.focus();
    send(b.dataset.cmd);
}));

// ----- speech in / o
const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
$('mic').onclick = () => {
    unlockSpeech();
    if (SR && window.isSecureContext) {

```

```

    if (listening && rec) { rec.stop(); return; }
    try {
        stopSpeaking();
        rec = new SR(); rec.lang = 'de-DE'; rec.interimResults = true; rec.continuo
        rec.onresult = (e) => { let t = ''; for (const r of e.results) t += r[0].tr
        rec.onend = () => { listening = false; $('mic').classList.remove('live'); i
        rec.onerror = (e) => { listening = false; $('mic').classList.remove('live')
        rec.start(); listening = true; $('mic').classList.add('live'); setState('li
        return;
    } catch (e) { /* fall back */ }
}
dictationHint();
};
function dictationHint() { $('text').focus(); $('reply').textContent = 'Tippe auf
let unlocked = false, voice = null;
function unlockSpeech() {
    unlockAudio();
    if (unlocked || !window.speechSynthesis) return;
    unlocked = true;
    const u = new SpeechSynthesisUtterance(' '); u.volume = 0; speechSynthesis.spea
}
function pickVoice() {
    if (!window.speechSynthesis) return null;
    const vs = speechSynthesis.getVoices().filter((v) => v.lang && v.lang.startsWit
    const male = ['Markus', 'Yannick', 'Martin', 'Viktor', 'Helmuth', 'Daniel'];
    return vs.find((v) => male.some((n) => v.name.includes(n)) && /premium|enhanced
}
if (window.speechSynthesis) speechSynthesis.onvoiceschanged = () => { voice = pic
let lastWhole = '';
function speak(text, whole) {
    if (!$('speak').checked || !window.speechSynthesis || !text) return;
    if (whole) { if (text === lastWhole) return; lastWhole = text; speechSynthesis.
    const clean = text.replace(/https?:\\\/\\\/S+/g, '').replace(/[*#`_]/g, '').replac
    if (!clean) return;
    const u = new SpeechSynthesisUtterance(clean);
    u.lang = 'de-DE'; u.voice = voice || pickVoice(); u.rate = 1.03; u.pitch = 0.9;
    u.onboundary = () => { level = 0.8; };
    speechSynthesis.speak(u);
}
function stopSpeaking() { lastWhole = ''; try { speechSynthesis.cancel(); } catch

// ----- views
let curView = 'core', lastTele = null;
function fileURL(p) { return `/api/file?path=${encodeURIComponent(p)}&t=${encodeURIComponent
function renderView(view, d) {
    curView = view;

```

```

const v = $('view');
let h = '';
if (view === 'globe') {
  h = `<div class="card"><h3>WELTLAGE <span>${esc(d.source || '')}</span></h3><
    (d.news || []).slice(0, 10).map((n) => `<a class="news" href="${esc(n.url)}
  } else if (view === 'markets') {
    const up = (d.changePct || 0) >= 0, an = d.analysis;
    h = `<div class="card"><h3>${esc(d.symbol || '')} <span>${esc(d.name || '')}<
      (an ? `<div class="sig ${an.signal === 'bullisch' ? 'up' : an.signal === 'b
        ((d.watch || []).length ? `<div class="list" style="margin-top:10px">' + d.
  } else if (view === 'weather') {
    const c = d.current || {}, dd = d.daily || {};
    h = `<div class="card"><h3>WETTER <span>${esc(d.place || '')}</span></h3><div
      `<div class="days">${(dd.time || []).slice(0, 7).map((t, i) => `<div>${new
  } else if (view === 'files') {
    h = `<div class="card"><h3>DATEIEN (MAC) <span>${esc(d.query || '')}</span></
  } else if (view === 'presentation') {
    h = `<div class="card"><h3>PRÄSENTATION</h3><div class="big" style="font-size
  } else if (view === 'document') {
    const s = d.spec || {};
    h = `<div class="card"><h3>DOKUMENT <span>${esc((d.format || '').toUpperCase(
  } else if (view === 'data') {
    const sh = (d.sheets || [d])[0] || {};
    h = `<div class="card"><h3>TABELLE <span>${esc(d.title || '')}</span></h3><di
  } else if (view === 'code') {
    h = `<div class="card"><h3>WEBSITE</h3><div class="muted">${esc(d.path || 'wi
  } else if (view === 'video') {
    h = `<div class="card"><h3>VIDEO <span>${d.rendering ? 'RENDERT ...' : esc(d.du
      (d.file && !d.rendering ? `<video src="${fileURL(d.file)}" controls playsin
      `<p>${esc(d.caption || '')}</p><div class="muted">${(d.hashtags || []).map(
        (d.file && !d.rendering ? `<button class="btn" id="copyCap">Beschreibung ko
  } else {
    curView = 'core';
    const t = lastTele;
    const mem = Memory.all();
    h = (t ? `<div class="card"><h3>MAC <span>${esc(t.host || '')}</span></h3><di
      `<div class="card"><h3>GEDÄCHTNIS (IPHONE) <span>${mem.length}</span></h3><
  }
v.innerHTML = h;
if (view === 'markets') drawChart($('chart'), (d.candles || []).map((c) => c.c)
const cc = $('copyCap');
if (cc) cc.onclick = () => { navigator.clipboard?.writeText(`${d.caption || ''}
}
function drawChart(cv, vals) {
  if (!cv || vals.length < 2) return;
  const dpr = devicePixelRatio || 1, w = cv.clientWidth, h = cv.clientHeight;

```

```

cv.width = w * dpr; cv.height = h * dpr;
const x = cv.getContext('2d'); x.scale(dpr, dpr);
const mn = Math.min(...vals), mx = Math.max(...vals), sp = (mx - mn) || 1;
const X = (i) => (i / (vals.length - 1)) * w, Y = (v) => 6 + (1 - (v - mn) / sp
const g = x.createLinearGradient(0, 0, 0, h); g.addColorStop(0, 'rgba(255,106,2
x.beginPath(); vals.forEach((v, i) => (i ? x.lineTo(X(i), Y(v)) : x.moveTo(X(i)
x.beginPath(); vals.forEach((v, i) => (i ? x.lineTo(X(i), Y(v)) : x.moveTo(X(i)
}

// ----- core animatio
(function core() {
  const cv = $('core'), x = cv.getContext('2d'), W = cv.width, c = W / 2;
  let t = 0;
  function frame() {
    t += 0.016;
    const speed = uiState === 'thinking' ? 3 : (uiState === 'speaking' || listeni
    level *= 0.92;
    x.clearRect(0, 0, W, W);
    const glow = x.createRadialGradient(c, c, 10, c, c, W / 2);
    glow.addColorStop(0, 'rgba(255,106,26,.35)'); glow.addColorStop(1, 'rgba(0,0,
    x.fillStyle = glow; x.fillRect(0, 0, W, W);
    [[150, 6, 1.2, '#ff6a1a'], [125, 2, -0.8, '#ffb066'], [175, 2, 0.5, 'rgba(255
    x.strokeStyle = col; x.lineWidth = lw;
    for (let s = 0; s < (k === 1 ? 1 : 3); s++) { const a = t * sp * speed + s
    });
    for (let i = 0; i < 60; i++) { const a = i / 60 * Math.PI * 2, r1 = 192, r2 =
    const pr = 46 + Math.sin(t * 3) * 3 + level * 30;
    x.fillStyle = brain.ready ? '#ff6a1a' : '#7a2e0c'; x.shadowColor = '#ff6a1a';
    x.shadowBlur = 0; x.fillStyle = brain.ready ? '#ffd6aa' : '#3a1a0a'; x.beginP
    requestAnimationFrame(frame);
  }
  frame();
})();

// ----- settings
function renderSettings() {
  $('models').innerHTML = Object.entries(MODELS).map(([k, m]) => `<label class="o
  document.querySelectorAll('input[name=model]').forEach((r) => (r.onchange = ()
  document.querySelectorAll('#mode button').forEach((b) => { b.classList.toggle('
}
$('gear').onclick = () => { renderSettings(); setBrainChip(); $('settings').hidde
$('closeSettings').onclick = () => { $('settings').hidden = true; };
$('loadBrain').onclick = () => { unlockSpeech(); loadBrain(); };

// ----- start
window.addEventListener('error', (e) => { const r = $('reply'); if (r && !r.textC

```

```
window.addEventListener('unhandledrejection', (e) => { const r = $('reply'); if (
if ('serviceWorker' in navigator && window.isSecureContext) navigator.serviceWork
renderView('core', {}));
setBrainChip();
setMac(false);
// Mac link only exists when the page is served by the Mac app (/api/ping answers
fetch('api/ping', { headers: token ? { Authorization: 'Bearer ' + token } : {} })
  .then((r) => { macHost = (r.headers.get('content-type') || '').includes('json')
  .catch(() => { macHost = false; })
  .finally(() => { setMac(false); connectMac(); });
if (store.get('jarvisBrainLoaded', '0') === '1' && params.get('engine') !== 'mock
window.JARVIS_MOBILE = { send, route, brain, made: () => made };
```